Task 4: Code Refactoring and Performance Optimization

Intern Name: Atharva J.

Project: Flask To-Do App

Language/Stack: Python, Flask, HTML, CSS

Project Overview:

The original project was a simple Flask To-Do web app with all logic in one file, no modular structure, and minimal validation or optimization.

Refactoring & Optimization Summary:

Area	Before	After
File Structure	All-in-one `app.py`	Separated into `routes`, `models`, `helpers`, etc.
Reusability	Repeated DB logic	Common logic moved to `helpers.py`
Variable Naming	Generic names	Meaningful names like `task_title`
Input Validation	None	Added blank input check
Performance	Used `.filter_by().first()`	Replaced with `.get()` for faster primary key lookup
Template Handling	Single template	Introduced `base.html` for reusability
CSS	Inline/missing	Moved to external `style.css`

New Project Structure:

```
todo-app/
 app/
    __init__.py
    models.py
    routes.py
    helpers.py
    templates/base.html
 static/style.css
 run.py
 requirements.txt
 refactor-report.md
```

Task 4: Code Refactoring and Performance Optimization

Why These Changes Matter:

- Improved maintainability, readability, and reusability.
- Faster database operations using `.get()` instead of `.filter_by().first()`.
- Added basic validation and user feedback.
- Clean modular architecture for scalability.

Functional Testing:

* Add, View, Toggle, and Delete Tasks
* Prevent empty tasks from being added
* Validated modular routing and data flow

Tools Used:

- Python 3, Flask, SQLite, HTML, CSS
- VS Code, GitHub

Conclusion:

The code is now scalable, readable, and professional, following industry-standard development practices.